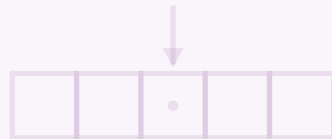


Лекция

6–7 апреля

Машина Тьюринга



Машина Тьюринга

Вычисление обычно выполняется записыванием определённых символов на бумаге.

— Алан Тьюринг

Чего не хватило автомату?

Язык $a^n b^n$ поставил конечные автоматы на место.

Лемма о накачке и теорема Майхилла–Нерода доказали: этот язык нерегулярен.

Причина честная: у автомата только конечное число состояний, и бесконечный счёт в них не помещается.

Люди считают: карандаш, бумага, стирание и переписывание.

Модель вычисления обязана уметь то же самое — запоминать неограниченно много.

Человек за столом

Тьюринг в 1936 году описал вычисление так, как его делает человек с карандашом:

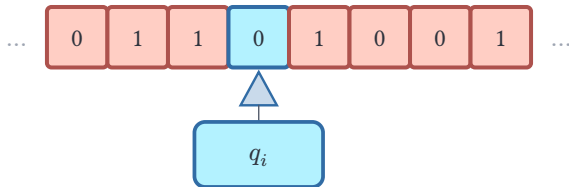
- конечное число «состояний души» — то, что человек помнит
- бумага, разбитая на клетки — то, куда он смотрит
- простые шаги — стереть, написать, сдвинуть взгляд.

Никакой магии: только символы, клетки и правила.

Всё остальное вычисление получает из их комбинаций.

Тьюринг описал вычислителя, а не вычислительную машину: под вопросом была проблема разрешения Гильберта, и чтобы доказать отсутствие алгоритма, алгоритм сначала нужно было определить.

Устройство машины



Три части с чёткими ролями: бесконечная лента — память, головка — взгляд, конечное управление — программа.

Бесконечность ленты — отсутствие предела, а не готовый склад: в каждый момент занята конечная часть, и любое вычисление касается лишь конечного числа ячеек.

Семёрка машины Тьюринга

Три части складываются в один формальный объект. Конечного в нём немного: состояния, алфавиты и таблица правил. Бесконечное ровно одно — лента, и именно в ней живёт весь неограниченный счёт.

Определение 1 (Машина Тьюринга)

Семёрка $M = (Q, \Gamma, \Sigma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$:

- Q — конечное множество состояний.
- Γ — алфавит ленты, содержит входной алфавит Σ и пробел.
- $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ — частичная функция переходов.
- q_0 — начальное состояние, q_{accept} и q_{reject} — принимающее и отвергающее.

Состояния конечны, лента — нет: весь неограниченный счёт живёт на ленте.

Компоненты: роли и границы

Компонент	Роль	Оговорка
Q	память между шагами: состояния помнят, что видели	конечна — счёт в состояниях не помещается
Γ, Σ	алфавит ленты с пробелом и входной алфавит	на входе пробелов нет, вокруг — пробелы
δ	пара (состояние, символ) даёт тройку	частична: нет правила — остановка
$q_0, q_{\text{accept}}, q_{\text{reject}}$	старт и два различных финала	финалы различны, иначе ответ без знака

Каждая оговорка — выбор: расширения не меняют класс, а частичность δ ловит молчание.

Один шаг

Вся работа машины складывается из атомарных шагов, и шаг — это всегда срабатывание одного правила.

Определение 2 (Переход)

На паре (состояние, символ) правило δ даёт тройку (новое состояние, новый символ, сдвиг): машина записывает новый символ под головкой, сдвигает головку влево или вправо и переходит в новое состояние.

δ частична: правила для текущей пары может не быть.

Остановка вне финальных состояний по соглашению считается отверганием.

Конфигурация

Определение 3 (Конфигурация)

Мгновенное описание работы — запись uqv :

- u — содержимое ленты слева от головки
- q — текущее состояние
- v — лента от головки вправо, включая читаемую ячейку.

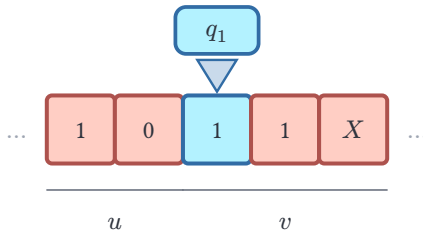
Начальная конфигурация на входе w — это q_0w : головка на первом символе.

Пример (Чтение конфигурации)

В записи $1q_0010$: $u = 1$, состояние q_0 , $v = 010$.

Головка стоит на первом символе v и читает 0.

Анатомия конфигурации



На схеме лента хранит $1011X$, головка стоит на третьей ячейке, и состояние q_1 нарисовано прямо над ней. Скобки внизу показывают, какие ячейки читаются как u и как v : слева от головки и от головки вправо. Пустые ячейки за X в запись не попадают, поэтому конфигурация всегда конечна, какой бы длиной ни была занятая часть ленты.

Три исхода

Определение 4 (Исходы работы)

- МТ **принимает** вход, если достигает q_{accept} .
- МТ **отвергает**, если достигает q_{reject} .
- МТ может **зациклиться** — не достигнуть ни того, ни другого.

Третий исход — не каприз определения, а суть модели.

Программа, которая крутится вечно, должна описываться честно.

У автомата два исхода: вход читается до конца, и ответ обязан появиться.

Машина Тьюринга добавляет молчание — ответа не будет ни на каком шаге.

Что значит решить язык?

Определение 5 (Распознаваемость и разрешимость)

Язык L **распознаваем**, если есть МТ, принимающая w в точности тогда, когда $w \in L$.

На $w \notin L$ машина может отвергнуть или заикнуться.

Язык L **разрешим**, если есть МТ, которая всегда останавливается: принимает $w \in L$, отвергает $w \notin L$.

Разрешимый язык распознаваем, а обратное — нет.

Классический пример — проблема остановки.

Пример: слова, заканчивающиеся на ноль

Пример (Семёрка в деле)

Язык слов над $\{0, 1\}$, заканчивающихся на 0.

Состояния: q_0 — «последний символ 1», q_1 — «последний символ 0».

- $(q_0, 1) \rightarrow (q_0, 1, R)$
- $(q_0, 0) \rightarrow (q_1, 0, R)$
- $(q_0, \text{пробел}) \rightarrow (q_{\text{reject}}, \text{пробел}, R)$
- $(q_1, 0) \rightarrow (q_1, 0, R)$
- $(q_1, 1) \rightarrow (q_0, 1, R)$
- $(q_1, \text{пробел}) \rightarrow (q_{\text{accept}}, \text{пробел}, R)$

Лента тут только возит головку: язык регулярный, память — два состояния, лента понадобится уже для подсчёта.

Полная трасса: шаг за шагом

Машина из примера выше на входе 1010: каждый шаг — одна строка.

Пример (Шаг за шагом)

Шаг	Конфигурация	Сработавшее правило
0	$q_0 1010$	$(q_0, 1) \rightarrow (q_0, 1, R)$: читаем 1, шаг вправо
1	$1q_0 010$	$(q_0, 0) \rightarrow (q_1, 0, R)$: ноль запомнен
2	$10q_1 10$	$(q_1, 1) \rightarrow (q_0, 1, R)$: единица сменила память
3	$101q_0 0$	$(q_0, 0) \rightarrow (q_1, 0, R)$: снова ноль в состоянии
4	$1010q_1$ пробел	$(q_1, \text{пробел}) \rightarrow (q_{\text{accept}}, \text{пробел}, R)$: край — принять

Конфигурация $q_0 1010$ переходит в $1q_0 010$ по одному правилу, и вся работа машины — цепочка таких переходов.

Трасса и разрешимость

Пример (Трасса на 10)

$$q_0 10 \rightarrow 1q_0 0 \rightarrow 10q_1 \square \rightarrow \text{принять.}$$

Последний шаг: q_1 видит пробел и принимает.

Машина читает $|w|$ символов, видит пробел и всегда останавливается.

Значит, язык разрешим — конечная память тут справляется без ленты.



Программы на ленте

Программы должны писаться для того, чтобы их читали люди, и лишь попутно — чтобы их исполняли машины.

— Гарольд Абельсон и Джеральд Сассман

Инверсия битов

Пример (Первый алгоритм)

Заменить каждый 0 на 1 и каждую 1 на 0.

Одно состояние, три правила:

- $(q_0, 0) \rightarrow (q_0, 1, R)$
- $(q_0, 1) \rightarrow (q_0, 0, R)$
- $(q_0, \text{пробел}) \rightarrow (q_{\text{accept}}, \text{пробел}, R)$

Эта машина вычисляет функцию, а не распознаёт язык: на ленте остаётся результат.

Разрешимость — про ответ «да или нет», вычислимость — про выход.

Правилам не нужно смотреть соседей: каждое решение принимается по паре (состояние, символ), и глобальный эффект складывается из локальных шагов.

Инкремент двоичного числа

Пример (Прибавить единицу)

Число на ленте, младший бит слева.

Пока читаем 1 — пишем 0 и уносим перенос вправо.

Первая 0 меняется на 1, и работа закончена.

- $(q_0, 1) \rightarrow (q_0, 0, R)$
- $(q_0, 0) \rightarrow (q_{\text{accept}}, 1, R)$
- $(q_0, \text{пробел}) \rightarrow (q_{\text{accept}}, 1, R)$

Лента — регистр неограниченной длины: новый разряд дописывается сам, и переполнения не бывает.

Инкремент: трасса

Пример (Трасса на 11)

1. $q_0 11$: читаем младшую 1, пишем 0, перенос вправо.
2. $0q_0 1$: снова 1, пишем 0, перенос вправо.
3. $00q_0 \square$: пробел — пишем 1 и принимаем.

На ленте 001: ровно $3 + 1$.

Все единицы исчерпаны — машина дописала новый разряд.

Лента сама расширяет число: ничего «перевыделять» не нужно.

Та самая $a^n b^n$

Пример (Вычёркивание пар)

Язык $a^n b^n$: конечному автомату не хватило на него состояний.

Стратегия: каждый проход вычёркивает один a слева и одну b справа.

Лента на входе $aabb$ после первого прохода и после второго:

$$aabb \rightarrow XabX \rightarrow XXXX.$$

Когда вся лента покрыта маркерами, машина видит пробел и принимает.

Конечному автомату не хватило состояний на счёт, а машине хватает ленты.

Сравнить два счётчика — значит стереть их попарно.

Таблица переходов: вычёркивание пар

Пример (Машина для $a^n b^n$)

Алфавит $\Gamma = \{a, b, X, \text{пробел}\}$, состояния: q_0 — поиск первого невычеркнутого, q_1 — проход вправо, q_2 — последняя невычеркнутая b с конца, q_3 — возврат к границе.

- $q_0: (q_0, X) \rightarrow (q_0, X, R), (q_0, a) \rightarrow (q_1, X, R), (q_0, b) \rightarrow (q_{\text{reject}}, b, R), (q_0, \text{пробел}) \rightarrow (q_{\text{accept}}, \text{пробел}, R)$
- $q_1: (q_1, a) \rightarrow (q_1, a, R), (q_1, b) \rightarrow (q_1, b, R), (q_1, X) \rightarrow (q_1, X, R), (q_1, \text{пробел}) \rightarrow (q_2, \text{пробел}, L)$
- $q_2: (q_2, X) \rightarrow (q_2, X, L), (q_2, b) \rightarrow (q_3, X, L), (q_2, a) \rightarrow (q_{\text{reject}}, a, L), (q_2, \text{пробел}) \rightarrow (q_{\text{reject}}, \text{пробел}, R)$
- $q_3: (q_3, a) \rightarrow (q_3, a, L), (q_3, b) \rightarrow (q_3, b, L), (q_3, X) \rightarrow (q_0, X, R)$

Три ловушки закрывают все несовпадения: лишняя b ловится в q_0 , a после b — в q_2 на символе, лишние a — поиском b до пробела.

Трасса: проход 1, шаги 0–4

Полная трасса на $aabb$, конфигурация за конфигурацией.

Пример (Вычеркнуть первый a и дойти до края)

Шаг	Конфигурация	Сработавшее правило
0	q_0 aabb	$(q_0, a) \rightarrow (q_1, X, R)$: вычеркнули первый a
1	Xq_1 abb	$(q_1, a) \rightarrow (q_1, a, R)$
2	$Xa q_1$ bb	$(q_1, b) \rightarrow (q_1, b, R)$
3	$Xab q_1$ b	$(q_1, b) \rightarrow (q_1, b, R)$
4	$Xabb q_1$ пробел	$(q_1, \text{пробел}) \rightarrow (q_2, \text{пробел}, L)$: край достигнут

Трасса: возврат, шаги 5–8

С конца вычёркивается последняя b , головка возвращается к границе.

Пример (Вычеркнуть последнюю b и вернуться)

Шаг	Конфигурация	Сработавшее правило
5	$Xab\ q_2\ b$	$(q_2, b) \rightarrow (q_3, X, L)$: вычеркнули последнюю b
6	$Xa\ q_3\ bX$	$(q_3, b) \rightarrow (q_3, b, L)$: возврат влево
7	$Xq_3\ abX$	$(q_3, a) \rightarrow (q_3, a, L)$
8	$q_3\ XabX$	$(q_3, X) \rightarrow (q_0, X, R)$: граница — к поиску

Трасса: проход 2, шаги 9–14

Осталось два невычеркнутых символа — один a и одна b .

Пример (Вычеркнуть оставшуюся пару)

Шаг	Конфигурация	Сработавшее правило
9	$Xq_0 \text{ abX}$	$(q_0, a) \rightarrow (q_1, X, R)$: второй проход начался
10	$XX q_1 \text{ bX}$	$(q_1, b) \rightarrow (q_1, b, R)$
11	$XXb q_1 \text{ X}$	$(q_1, X) \rightarrow (q_1, X, R)$
12	$XXbX q_1 \text{ пробел}$	$(q_1, \text{пробел}) \rightarrow (q_2, \text{пробел}, L)$
13	$XXb q_2 \text{ X}$	$(q_2, X) \rightarrow (q_2, X, L)$: вычеркнутый хвост
14	$XX q_2 \text{ bX}$	$(q_2, b) \rightarrow (q_3, X, L)$: последняя b вычеркнута

Трасса: приём, шаги 15–17

Пример (Приём)

Шаг	Конфигурация	Сработавшее правило
15	XX q_3 XX	$(q_3, X) \rightarrow (q_0, X, R)$
16	XXX q_0 X	$(q_0, X) \rightarrow (q_0, X, R)$
17	XXXX q_0 пробел	$(q_0, \text{пробел}) \rightarrow (q_{\text{accept}}, \text{пробел}, R)$: всё вычеркнуто
18	XXXX q_{accept}	принято: $n = 2$, как и было

Каждый проход уменьшает невычеркнутую часть ровно на два символа, поэтому работа кончается: либо приём, либо одно из отвергающих правил.

Остановка гарантирована на любом входе — таблица описывает решатель.

Палиндромы

Пример (Сравнение концов)

Палиндром над $\{a, b\}$ одинаково читается в обе стороны, например $abba$ и $baab$.

Стратегия: сравнить крайние символы, стереть оба и повторять, пока строка не кончится.

Состояния: q_0 читает с начала, q_a и q_b несут символ вправо, p_a и p_b сверяют его с правым краем, q_{back} возвращает головку.

Несовпадение видно сразу, вход ab :

$$q_0 \text{ } ab \rightarrow Xq_a \text{ } b \rightarrow Xb \text{ } q_a \text{ пробел} \rightarrow Xp_a \text{ } b \rightarrow Xq_{reject} \text{ } b.$$

МТ видит оба конца строки ценой многократных проходов: запоминание через состояние — то же, что у автомата, но возвраты делает головка.

Палиндром: проход 1

Полный прогон на abba: каждый шаг — одна строка, маркер X помечает стёртую ячейку.

Пример (Запомнить a и сверить правый край)

Шаг	Конфигурация	Сработавшее правило
0	q_0 abba	$(q_0, a) \rightarrow (q_a, X, R)$: a стёрт и запомнен
1	Xq_a bba	$(q_a, b) \rightarrow (q_a, b, R)$
2	$Xb q_a$ ba	$(q_a, b) \rightarrow (q_a, b, R)$
3	$Xbb q_a$ a	$(q_a, a) \rightarrow (q_a, a, R)$
4	$Xbba q_a$ пробел	$(q_a, \text{пробел}) \rightarrow (p_a, \text{пробел}, L)$: край — развернуться
5	$Xbb p_a$ a	$(p_a, a) \rightarrow (q_{\text{back}}, X, L)$: край совпал — стереть
6	$Xb q_{\text{back}}$ bX	$(q_{\text{back}}, b) \rightarrow (q_{\text{back}}, b, L)$: возврат влево

Палиндром: возврат и второй заход

q_{back} доходит до границы, q_0 стирает следующий символ, и машина сверяет второй край.

Пример (Стереть b и развернуться у правого края)

Шаг	Конфигурация	Сработавшее правило
7	$Xq_{\text{back}} \text{bbX}$	$(q_{\text{back}}, b) \rightarrow (q_{\text{back}}, b, L)$
8	$q_{\text{back}} \text{XbbX}$	$(q_{\text{back}}, X) \rightarrow (q_0, X, R)$: граница — к чтению
9	$Xq_0 \text{bbX}$	$(q_0, b) \rightarrow (q_b, X, R)$: b стёрт и запомнен
10	$\text{XX } q_b \text{ bX}$	$(q_b, b) \rightarrow (q_b, b, R)$
11	$\text{XXb } q_b \text{ X}$	$(q_b, X) \rightarrow (q_b, X, R)$: мимо стёртого хвоста
12	$\text{XXbX } q_b \text{ пробел}$	$(q_b, \text{пробел}) \rightarrow (p_b, \text{пробел}, L)$: край — развернуться
13	$\text{XXb } p_b \text{ X}$	$(p_b, X) \rightarrow (p_b, X, L)$: пропуск стёртых

Палиндром: приём

Нечётный палиндром кончается так же: последний символ стирается, приём — на пробеле.

Пример (Второе сравнение и приём)

Шаг	Конфигурация	Сработавшее правило
14	$XX p_b bX$	$(p_b, b) \rightarrow (q_{\text{back}}, X, L)$: край совпал — стереть
15	$X q_{\text{back}} XXX$	$(q_{\text{back}}, X) \rightarrow (q_0, X, R)$: граница — к чтению
16	$XX q_0 XX$	$(q_0, X) \rightarrow (q_0, X, R)$: пропуск маркеров
17	$XXX q_0 X$	$(q_0, X) \rightarrow (q_0, X, R)$
18	$XXXX q_0 \text{ пробел}$	$(q_0, \text{ пробел}) \rightarrow (q_{\text{accept}}, \text{ пробел}, R)$: всё стёрто
19	$XXXX q_{\text{accept}}$	принято: abba — палиндром

Одна головка платит за пару двумя проходами: $O(n^2)$ шагов, многоленточная машина — $O(n)$.

Приёмы программирования

Примеры собираются из повторяющихся приёмов.

Приём	Идея
Запоминание через состояние	«вижу символ a » — перейди в состояние «помню a » и носи его, пока не понадобится
Маркеры	обработанные символы заменяются на X : «уже учтено» отделяется от «ещё нет»
Проход к краю и возврат	«иди вправо, пока не пробел», шаг влево, переключение — цикл с условием на краю

Счёт на МТ — это всегда расстановка и пересчёт маркеров.

Память машины растёт не числом состояний, а занятой частью ленты.

Машина как процессор

Приёмы — прообраз настоящего процессора:

- состояние играет роль счётчика команд
- маркер — флага
- проход по ленте — цикла.

Программа на МТ — конечная таблица, и её можно записать строкой символов.

Композиция машин

Машины собираются друг из друга: выход одного блока — вход следующего.

Определение 6 (Композиция)

Если M_1 переводит ленту из u в v , а M_2 — из v в w , состояние останова M_1 отождествляется с начальным состоянием M_2 .

Из блоков собираются большие программы.

Пример (Собранная машина)

Удвоение унарного числа, затем инверсия битов: составная машина вычисляет $1^n \rightarrow 0^{2n}$.

На входе 11 удвоитель даёт 1111, инвертор — 0000.

Останов составной машины — останов последнего блока.

Машина как данные

Правило перехода — это данные: пара на входе, тройка на выходе. Таблицу переходов можно положить в обычную структуру данных.

Пример (Таблица на Rust)

```
fn step(q: &str, a: char) -> Option<(char, isize, &'static str)> {  
    match (q, a) {  
        ("q0", '1') => Some(('1', 1, "q0")),  
        ("q0", '0') => Some(('0', 1, "q1")),  
        ("q0", '#') => Some(('#', 0, "rej")),  
        ("q1", '1') => Some(('1', 1, "q0")),  
        ("q1", '0') => Some(('0', 1, "q1")),  
        ("q1", '#') => Some(('#', 0, "acc")),  
        _ => None,  
    }  
}
```

Интерпретатор: цикл с головкой

Пример (Печать конфигурации *uqv* на каждом витке)

```
fn main() {  
    let mut tape: Vec<char> = "1010#".chars().collect();  
    let (mut q, mut i) = ("q0", 0);  
    while q != "acc" && q != "rej" {  
        let u: String = tape[..i].iter().collect();  
        let v: String = tape[i..].iter().collect();  
        println!("{}", format!("{u} {q} {v}").trim_start());  
        let (b, d, q2) = step(q, tape[i]).expect("нет правила");  
        tape[i] = b;  
        i = (i as isize + d) as usize;  
        q = q2;  
    }  
}
```

Что печатает симулятор?

Пример (Запуск на 1010)

```
q0 1010#  
1 q0 010#  
10 q1 10#  
101 q0 0#  
1010 q1 #
```

Пять строк — пять шагов таблицы переходов, и цикл вышел в асс.

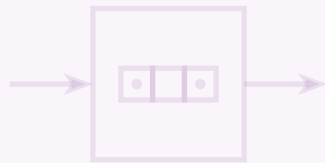
Роль головки играет индекс i , а символ $\#$ изображает пробел: у вектора нет бесконечности, край помечен явно.

Программа как конечная строка

Таблица переходов — конечный объект: её можно выписать строкой символов.

Значит, программу можно положить на ленту и читать её как данные.

Одна машина, которая делает это по правилам, исполнила бы любую программу.



Универсальность и устойчивость

Только тьюринговская работа дала точную и несомненно адекватную дефиницию формальной системы.

— Курт Гёдель

Программа как строка

Пример (Кодировка $\langle M \rangle$)

Состояния нумеруются: $q_0 = 1$, $q_{\text{accept}} = 2$, $q_{\text{reject}} = 3$.

Символы и направления — следующие числа: $0 = a$, $1 = b$, «пробел» = c , $L = l$, $R = r$.

Переход $\delta(q, a) = (q', b, D)$ — пятёрка чисел, таблица — пятёрки подряд.

Инкрементная машина даёт строку $\langle M \rangle = (1, b, 1, a, r)(1, a, 2, b, r)(1, c, 2, b, r)$.

Раз машина — строка, программа превращается в данные: её можно положить на ленту, скопировать, передать другой машине.

Некорректные строки по договорённости — код машины, которая на любом входе сразу отвергает, так что каждый текст описывает хоть какую-то машину.

Машины можно перенумеровать

Теорема 1 (Счётность машин)

Машины Тьюринга образуют счётное множество: они выписываются в список $M_1, M_2, M_3, \dots$ без пропусков.

Доказательство

Докажем перечислением строк.

Строки перечисляются по возрастанию длины, внутри длины — лексикографически.

Корректные строки дают машины списка, некорректные — фиктивные машины, так что пропускать нечего.

Годится любая нумерация, порождаемая алгоритмом: аргумент опирается на существование списка.

Универсальная машина

Теорема 2 (Универсальная машина)

Существует машина U , которая на входе $\langle M \rangle \# w$ принимает, отвергает и закичивается ровно тогда, когда это делает M на слове w .

Доказательство

Докажем построением симулятора с тремя дорожками: программа $\langle M \rangle$, текущая лента M , состояние и маркер позиции головки.

Один шаг U : прочитать символ под маркером, найти в таблице пятёрку с этой парой, переписать ячейку, сдвинуть маркер, записать новое состояние.

U конечна и одна на все машины: интерпретаторы, виртуальные машины и эмуляторы — её инженерные потомки.

Один шаг универсальной машины

Пример (Шаг симуляции)

Пусть M — инкремент, вход 11.

На ленте U лежит $\langle M \rangle \# 11 \# 1$: программа, слово, номер текущего состояния.

U читает на дорожке работы первый символ 1, берёт состояние 1 и находит пятёрку $(1, b, 1, a, r)$: записать 0, сдвиг вправо.

Через три таких шага дорожка работы показывает 001 — ровно то, что оставляла бы сама M .

Симуляция точна: U повторяет каждый шаг M , а не только результат.

Медленнее — да, слабее — нет.

Эквивалентность вариантов

Определение содержит произвольные на вид решения: две стороны бесконечности, сдвиг ровно на клетку, один исход на шаг.

Теорема 3 (Устойчивость класса)

Односторонняя лента, сдвиг S (остаться на месте), k лент с k головками и недетерминированный выбор не расширяют класс распознаваемых языков: каждую такую машину симулирует обычная семёрка.

Определение МТ — не догадка об устройстве настоящей машины, а точка схождения: меняй удобства, класс языков прежний.

Как устроена симуляция?

Доказательство

Докажем симуляцией, вариант за вариантом.

Односторонняя лента. Правая половина на верхнюю дорожку, левая на нижнюю отражённо.

Сдвиг на месте. Переход через свежее состояние: влево с записью b , затем вправо.

Много лент. Ленты — столбиками с маркерами головок, шаг — два прохода по занятой части.

Недетерминизм. Дерево конфигураций обходится в ширину, по шагу на ветвь.

«Дорожки» — не новая конструкция: ячейка хранит пару символов, алфавит-произведение конечен.

Цена симуляции

Вариант	Цена симуляции	Откуда берётся
односторонняя лента	постоянный множитель	две полуленты на двух дорожках
сдвиг S	2 шага за 1	заход влево и возврат вправо
k лент	$O(t^2)$ шагов за t шагов	занятая часть растёт на константу за шаг, и каждый шаг — проход по ней
недетерминизм	до b^t конфигураций	обход дерева в ширину: ветви длины до t

Эквивалентность говорит, что решить можно, и молчит, насколько это дорого: вопрос «можно ли без экспоненты» — главный вопрос теории сложности.

Недетерминированная машина — не многопроцессорный компьютер: ветвление — абстракция «угадал и проверь», симулятор исполняет ветви по очереди.

Тезис Чёрча–Тьюринга

Все варианты сошлись к одной машине.

Осталось спросить, измеряет ли она вычислимость вообще.

Тезис Чёрча–Тьюринга: всякая функция, вычисляемая каким угодно механическим процессом с конечным описанием, вычислима на машине Тьюринга.

Статус — не теорема: доказательство связывает два формальных объекта, а здесь одна сторона, «вычислимо интуитивно», формальным объектом не является.

Статус — определение, проверяемое опытом: формализмы сошлись к одному классу, а процедура вне машинного класса за девяносто лет не предъявлена.

Свидетельства и следствие

Формализм	Автор, год	Чем вычисляет
λ -исчисление	Чёрч, 1936	подстановка термов вместо ленты
общерекурсивные функции	Гёдель–Клини, 1936	композиция и минимизация
машины Поста	Пост, 1936	продукции переработки слов
регистровые машины	Минский, 1961	арифметика над целыми регистрами
языки программирования	с 1950-х	интерпретатор сам симулируется на МТ

Рабочее следствие тезиса: неразрешимость, доказанная для МТ, закрывает задачу для всех языков и машин сразу.

Полнота конкретного языка — уже теорема, а не тезис: интерпретатор МТ пишется на этом языке, и симуляция строится явно.

Проблема остановки

Тестирование показывает наличие ошибок, но не может доказать их отсутствие.

— Эдсгер Дейкстра



Диагональная таблица

Строки — машины M_i , столбцы — входы w_j , значения клеток для примера.

Машина	1	10	11	100	101
M_1	+	—	«ЦИКЛ»	+	—
M_2	«ЦИКЛ»	+	+	«ЦИКЛ»	«ЦИКЛ»
M_3	+	—	+	+	—
M_4	«ЦИКЛ»	+	—	—	+
M_5	—	«ЦИКЛ»	+	«ЦИКЛ»	+

+ — принял, — — отверг, «ЦИКЛ» — не остановился.

Диагональные клетки задают язык: w_i лежит в D , когда M_i не принимает w_i .

Инвертированная диагональ — таблица, с которой не совпадает ни одна строка.

Диагональный язык

Теорема 4 (Диагональный язык D)

Язык $D = \{w_i \mid M_i \text{ не принимает } w_i\}$ не распознаваем никакой машиной Тьюринга.

Доказательство

Докажем от противного: пусть M_k распознаёт D .

Спросим о слове w_k — входе с номером, равным номеру машины.

По определению D : $w_k \in D \leftrightarrow M_k$ не принимает w_k .

По свойству распознавателя: $w_k \in D \leftrightarrow M_k$ принимает w_k .

Собирая: M_k принимает w_k в точности когда не принимает — противоречие.

Машин счётное число, языков несчётное: диагональ предъявляет один из лишних.

Проблема остановки

Диагональ показала, что языки без распознавателя существуют.

Определение 7 (Проблема остановки)

$\text{HALT} = \{\langle M \rangle \# w \mid M \text{ останавливается на } w\}$: по коду машины и входу узнать, остановится ли вычисление.

HALT распознаваем: симуляция U — и есть распознаватель, она принимает, если M остановилась.

Честная оговорка: на неостанавливаемом входе симулятор молчит вечно, ответа «нет» он не даёт.

«Подождать подольше» не помогает: нет числа шагов, после которого позволено сказать «не остановится», — это свойство модели.

Неразрешимость

Теорема 5 (Тьюринг, 1936)

Проблема остановки не разрешима: не существует машины, которая на каждой паре $\langle M \rangle, w$ останавливается и верно отвечает, остановится ли M на w .

Запрещён не симулятор — симуляция работает.

Запрещён всегда останавливающийся безошибочный предсказатель остановки.

Доказательство: от противного

Доказательство

Докажем от противного: пусть решатель H всегда останавливается и верно отвечает 1 (остановится) или 0 (нет) на каждой паре $\langle M \rangle, w$.

Построим машину D : на входе $\langle M \rangle$ она запускает H на $\langle M \rangle, \langle M \rangle$ и поступает наоборот — при ответе 1 вечный цикл, при 0 останов.

Спросим о D её собственный код: что делает D на входе $\langle D \rangle$?

Случай 1. D остановился — значит, H ответил 1 и D уходит в цикл. Противоречие.

Случай 2. D не остановился — значит, H ответил 0 и D по построению останавливается.

Оба исхода невозможны, значит, H с заявленными свойствами не существует.

Оговорки и цена вопроса

Построение D — диагональ: H применяется к паре, где слева и справа один и тот же код, — шаг по диагонали таблицы.

Машина D не запрещена как таковая: она определена через H , и уничтожено лишь допущение «решатель существует».

Итог по языкам: HALT распознаваем, но не разрешим.

Дополнение $\overline{\text{HALT}}$ даже не распознаваемо.

Инженерное эхо: анализатора, который для любой программы гарантированно решает, зависнет ли она, не существует — отсюда таймауты, watchdog'и и лимиты ресурсов.

Что мы поняли: машина Тьюринга

Автомат получил ленту и стал универсальным, но универсальность немедленно показала и предел. Диагональ не распознаётся ни одной машиной, а проблема остановки распознаваема, но не разрешима.

- МТ — бесконечная лента плюс конечное управление: неограниченная память при финитном описании.
- Конфигурация uqv — полный снимок работы, вычисление — цепочка конфигураций, исходов три.
- Программы пишутся приёмами: символ в состоянии, маркеры, проходы, композиция — лента заменяет счётчик, и $a^n b^n$ разрешимо.

Программа как данные и граница машин

Машина оказалась строкой, строка — данными на ленте, и универсальная машина исполнила любую программу. На этой же идее держится главный негативный результат.

- Машина — строка $\langle M \rangle$, машины счётны, а универсальная U исполняет любую: программа стала данными.
- Варианты — односторонняя лента, k лент, недетерминизм — не меняют силу, только цену: $O(t^2)$ и b^t .
- Тезис Чёрча–Тьюринга — проверенное опытом определение вычислимости, а не теорема.
- Диагональ даёт язык без распознавателя, а HALT распознаваем, но не разрешим.

Вопросы для самопроверки

Вопросы для самопроверки

1. Почему $a^n b^n$ нерегулярен, но разрешим машиной Тьюринга?
2. Назовите все семь компонентов семёрки МТ и роль каждого.
3. Что записывает конфигурация uqv и как выглядит начальная конфигурация для входа 10 ?
4. Чем распознаваемость языка отличается от разрешимости?
5. Как устроена кодировка $\langle M \rangle$ и почему машины можно выстроить в список?
6. Что делает универсальная машина и какие три дорожки она ведёт?
7. Во сколько обходится симуляция k -ленточной машины и откуда берётся множитель?
8. Воспроизведите доказательство неразрешимости HALT: что такое H и D , где возникает противоречие?

Что почитать

- m25 — «Определение машины Тьюринга», «Примеры машин Тьюринга», «Стандартные приёмы программирования», «Устойчивость и варианты», «Тезис Чёрча–Тьюринга», «Универсальная машина Тьюринга».